

REFERENCES

- Vahid Behzadan and Arslan Munir. Vulnerability of deep reinforcement learning to policy induction attacks. *arXiv preprint arXiv:1701.04143*, 2017.
- Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *J. Artif. Intell. Res.(JAIR)*, 47:253–279, 2013.
- Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*, 2015.
- Alhussein Fawzi, Omar Fawzi, and Pascal Frossard. Analysis of classifiers’ robustness to adversarial perturbations. *Machine Learning*, 107(3):481–508, 2018.
- Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. *arXiv preprint arXiv:1706.04599*, 2017.
- Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Sandy Huang, Nicolas Papernot, Ian Goodfellow, Yan Duan, and Pieter Abbeel. Adversarial attacks on neural network policies. *arXiv preprint arXiv:1702.02284*, 2017.
- Jernej Kos and Dawn Song. Delving into adversarial attacks on deep policies. *arXiv preprint arXiv:1705.06452*, 2017.
- Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. 2009.
- Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world. *arXiv preprint arXiv:1607.02533*, 2016.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Aran Nayebi and Surya Ganguli. Biologically inspired protection of deep networks from adversarial attacks. *arXiv preprint arXiv:1703.09202*, 2017.
- Martin J Osborne and Ariel Rubinstein. *A course in game theory*. MIT press, 1994.
- Nicolas Papernot and Patrick McDaniel. On the effectiveness of defensive distillation. *arXiv preprint arXiv:1607.05113*, 2016.
- Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pp. 506–519. ACM, 2017.
- Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of machine learning research*, 15(1):1929–1958, 2014.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.

Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. *arXiv preprint arXiv:1705.07204*, 2017a.

Florian Tramèr, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. The space of transferable adversarial examples. *arXiv preprint arXiv:1704.03453*, 2017b.

A REINFORCEMENT LEARNING MODEL ARCHITECTURE

For the experiments in section 4.2, we trained the network with RMSProp, minibatches of size 32, a learning rate of 0.00025, and a momentum of 0.95 and as in (Mnih et al., 2015) where the discount factor is $\gamma = 0.99$, the number of steps between target updates to 10000 steps. We updated the network every 4 steps by randomly sampling a minibatch of size 32 samples from the replay buffer and trained the agents for a total of $100M$ steps per game. The experience replay contains the $1M$ most recent transitions. For training we used an ε -greedy policy with ε annealed linearly from 1 to 0.1 over the first $1M$ time steps and fixed at 0.1 thereafter.

The input to the network is $4 \times 84 \times 84$ tensor with a rescaled, gray-scale version of the last four observations. The first convolution layer has 32 filters of size 8 with a stride of 4. The second convolution layer has 64 filters of size 4 with stride 2. The last convolution layer has 64 filters of size 3 followed by two fully connected layers with size 512 and the final fully connected layer Q-value of each action where *ReLU* rectifier is deployed for the nonlinearity at each layer.

B OTHER METHODS

We tried a variety of different methods that could be added to pretrained models and tested their performance against adversarial examples. The following is a continuation of section 4.1, where we use the dense model again on the CIFAR-10 dataset.

B.1 RANDOM NOISY WEIGHTS (RNW)

One simple way of introducing stochasticity to the activations is by adding random Gaussian noise to each weight, with mean 0 and constant standard deviation, s . So each weight tensor W^i now changes to $M(W^i)$, where the j 'th entry is given by

$$M(W^i)_j = (W^i)_j + \eta, \quad \eta \sim \mathcal{N}(0, s^2).$$

These models behave very similar to the dense model (Fig. 5a, the legend indicates the value of s). While we test several different values of s , we do not observe any significant improvements regarding robustness against adversarial examples. As s increased, the accuracy for non-zero λ decreased.

B.2 RANDOMLY SCALED WEIGHTS (RSW)

Instead of using additive noise, we also try multiplicative noise. The scale factor can be picked from a Gaussian distribution, with mean 1 and constant standard deviation s . So each weight tensor W^i now changes to $M(W^i)$, where the j 'th entry is given by

$$M(W^i)_j = \eta \cdot (W^i)_j, \quad \eta \sim \mathcal{N}(1, s^2).$$

These models perform similar to the dense model, but again, no robustness is offered against adversarial examples. They follow a similar trend as the RNW models (Figure 5b, the legend indicates the value of s).

B.3 DETERMINISTIC WEIGHT PRUNING (DWP)

Following from the motivation of preventing perturbations to propagate forward in the network, we tested deterministic weight pruning, where the top $k\%$ entries of a weight matrix were kept, while the rest were pruned to 0, according to their absolute values. This method was prompted by the success achieved by this pruning method, introduced by Han et al. (2015), where they also fine-tuned the model.

For low levels of pruning, these models do very similar to the dense model, even against adversarial examples (Fig. 5c, the legend indicates the value of k). The adversary can compute the gradient of the sparse model, and the perturbations propagate forward through the surviving weights. For higher levels of sparsity, the accuracy in the no-perturbation case drops down quickly.